🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 15: Fuzzing an Application with American Fuzzy Lop (AFL) and Capstone Framework
```

Scenario

Fast In-Process Fuzzing with AFL and Capstone combines the Capstone disassembly engine library (`libcapstone.a`) with LLVM-based compiler wrappers (**afl-clang-fast**) and in-process persistent looping macros (**__AFL_LOOP**) to achieve extreme execution speeds (>12,000 execs/sec) during automated vulnerability discovery.

Key technical concepts in persistent in-process fuzzing:
1. **Capstone Engine Disassembly**: Disassembling raw binary instruction byte streams using Capstone API routines.
2. **LLVM Compiler Instrumentation**: Using **afl-clang-fast** statically linked against `libcapstone.a` (`afl-clang-fast -static captest.c capstone/libcapstone.a -o captest`) to achieve fine-grained code coverage tracking without fork overhead.
3. **Persistent In-Process Looping**: Standard AFL forks a new process for every test case input (limiting speed to ~2,000 execs/sec). Incorporating the macro **while (__AFL_LOOP(1000))** in **captest2.c** feeds 1,000 mutated inputs per process lifecycle before resetting, increasing execution speed to **12.8k/sec**.
4. **Kernel Core Pattern Configuration**: Resetting `/proc/sys/kernel/core_pattern` via **echo > /proc/sys/kernel/core_pattern** to prevent external crash handling processes from throttling AFL fuzzing loops.

In this exercise, we log into **Ubuntu 20.04 Fuzzing-CAP** VM (**student** / **studentpassword**), inspect base Capstone C test file **captest.c**, compile static binary using `afl-clang-fast` , generate pseudo-random seeds via `dd` , reset kernel core pattern, run initial AFL fuzzing loop (observing ~2k execs/sec), compile persistent loop version **captest2.c**, and achieve **12.8k/sec** execution speed.

Objective

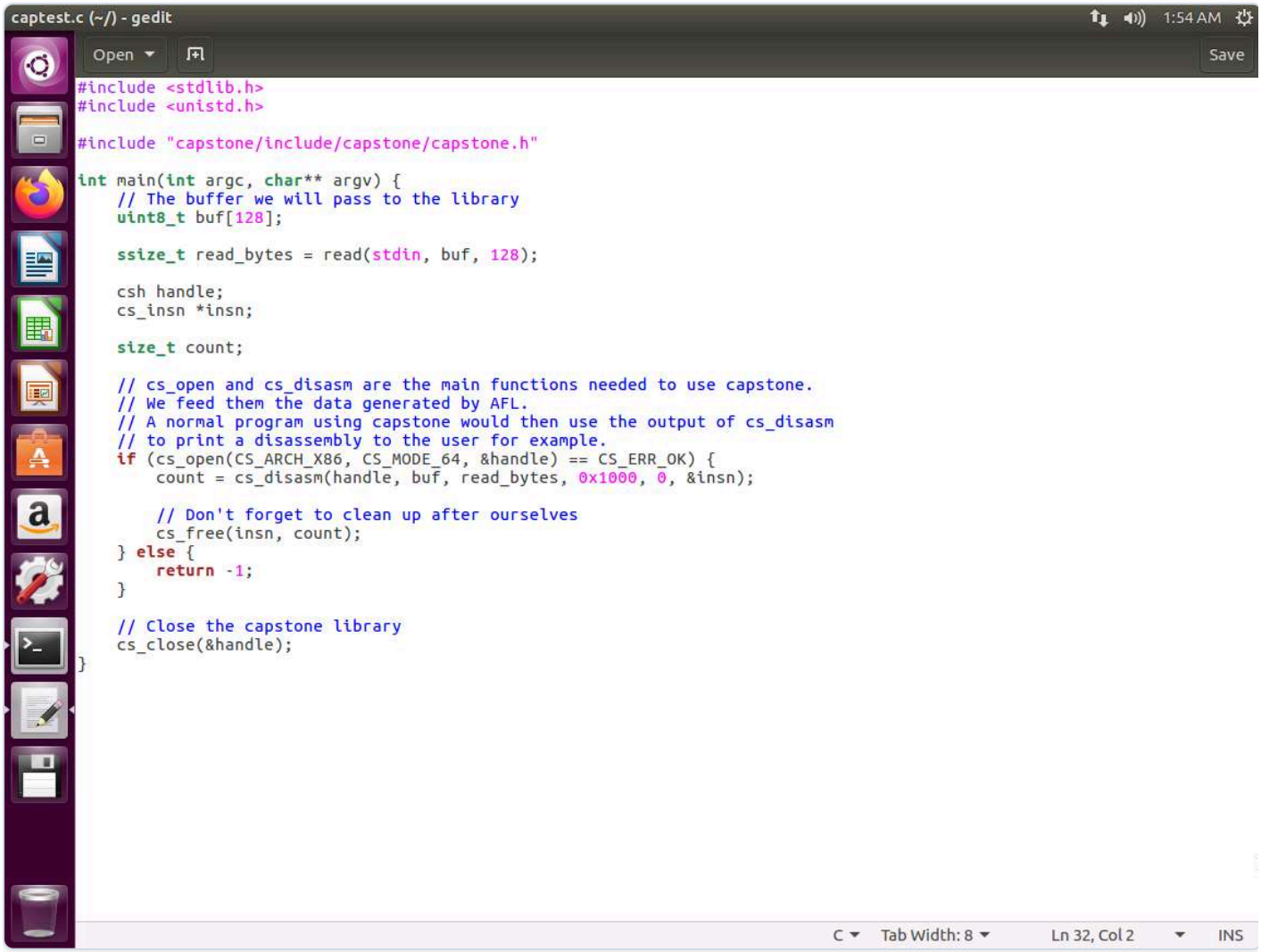
The objective of this lab is to help students learn how to:
- Authenticate to Ubuntu 20.04 Fuzzing-CAP target VM (**student** / **studentpassword**)
- Inspect Capstone disassembly test harness source (**captest.c**)
- Compile static instrumented binary using LLVM wrapper: **afl-clang-fast -static captest.c capstone/libcapstone.a -o captest**
- Generate pseudo-random input seeds: **dd if=/dev/urandom of=inputs/rand bs=64 count=1**
- Reset core dump pattern: **echo > /proc/sys/kernel/core_pattern**
- Run baseline AFL fuzzing session: **afl-fuzz -i inputs -o findings ./captest**
- Implement persistent in-process fuzzing macro **__AFL_LOOP(1000)** in **captest2.c**
- Compile in-process target binary: **afl-clang-fast -static captest2.c capstone/libcapstone.a -o captest2**
- Execute optimized persistent fuzzing session: **afl-fuzz -i inputs -o findings ./captest2**
- Measure execution speed optimization jump from ~2k/sec to **12.8k/sec**

Task 1: Compile Capstone Harness, Run AFL Baseline, Implement Persistent Looping, and Measure Exec Speed

In this task, we inspect `captest.c`, compile with `afl-clang-fast`, generate seed inputs, configure kernel core patterns, run AFL baseline fuzzing, implement `__AFL_LOOP(1000)` in `captest2.c`, and achieve 12.8k/sec exec speed.

Lab Tasks

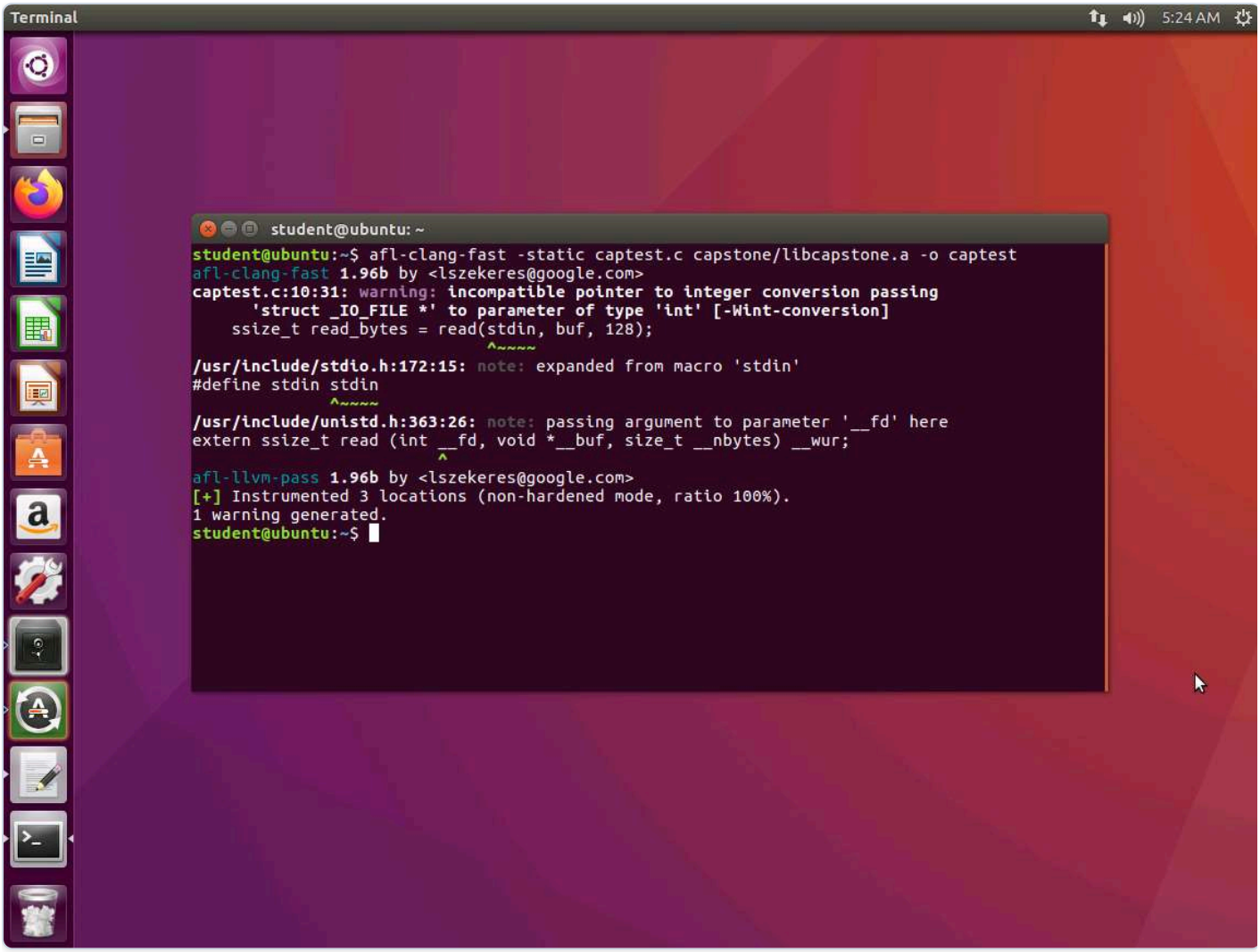
1. Log into **Ub20 Fuzzing-CAP** VM using credentials (**student** / **studentpassword**). Inspect Capstone test code **captest.c**.



1. Review optional compilation commands reference.

```
# compile an instrumented version of capstone
git clone https://github.com/aquynh/capstone.git
cd capstone
CC=afl-clang-fast ./make.sh
```

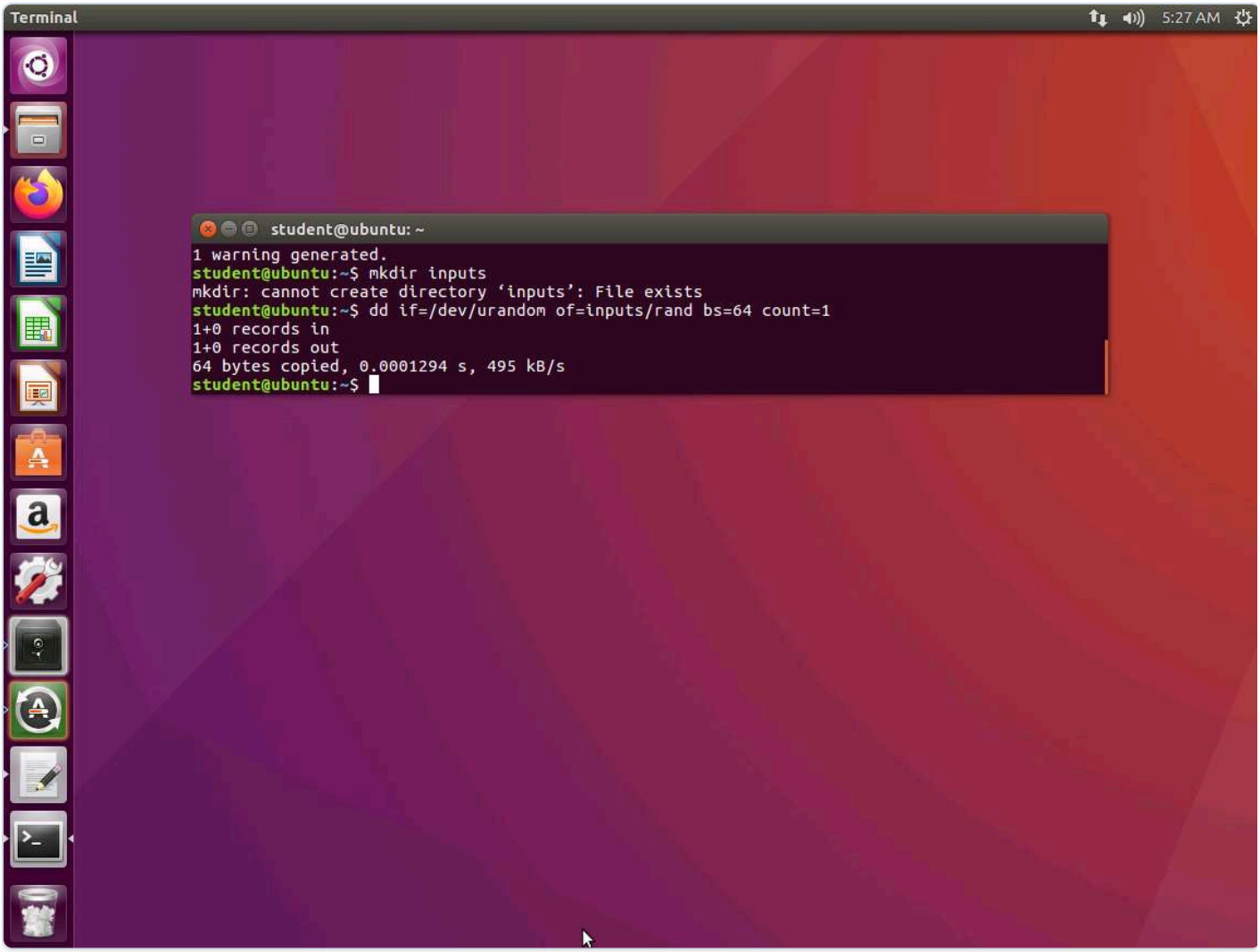
1. Compile static test binary using `afl-clang-fast` and `libcapstone.a`:
   afl-clang-fast -static captest.c capstone/libcapstone.a -o captest



1. Create input directory and generate 64-byte pseudo-random seed:

   mkdir inputs

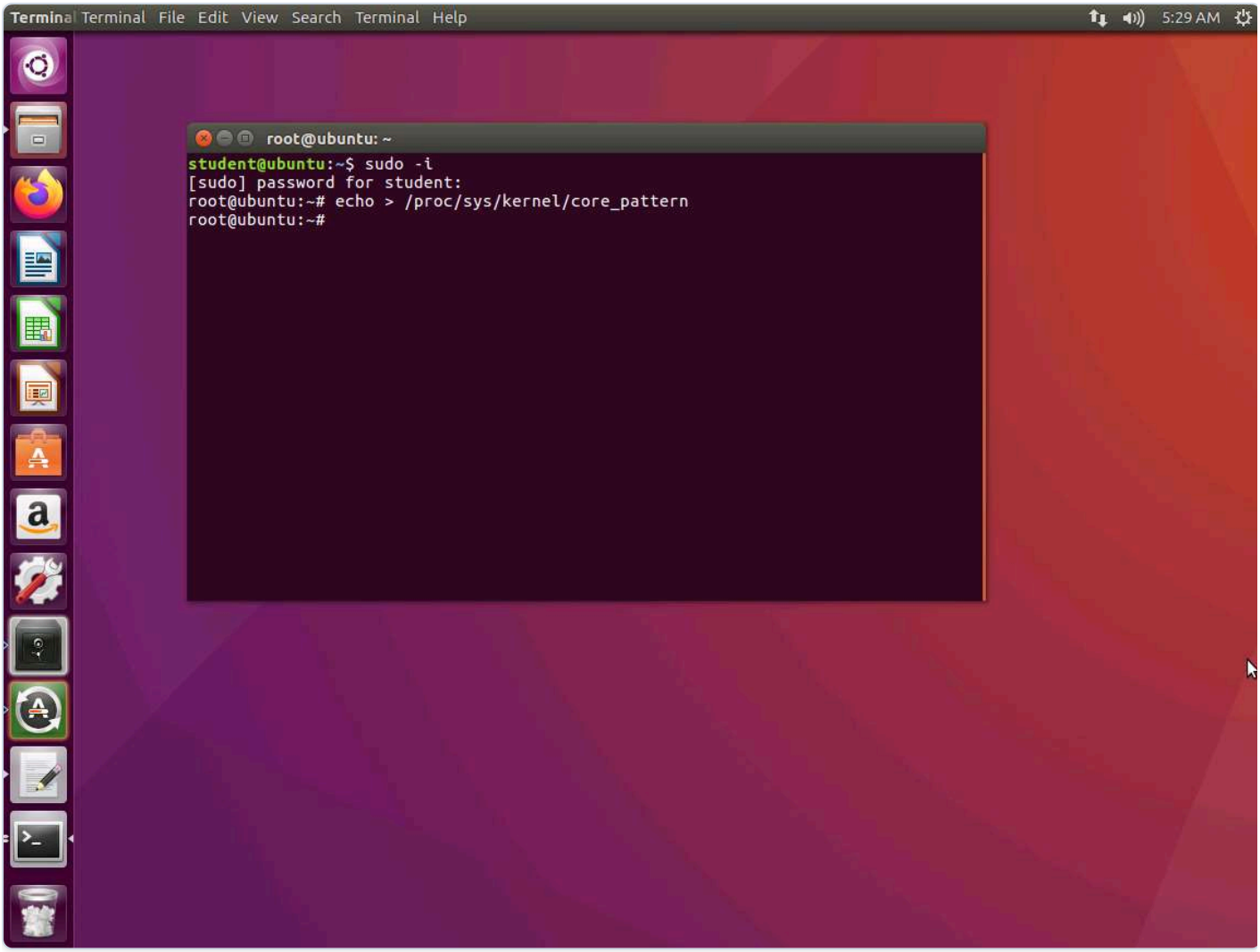
   dd if=/dev/urandom of=inputs/rand bs=64 count=1



1. Open root terminal shell and clear core dump pattern:

   sudo -i

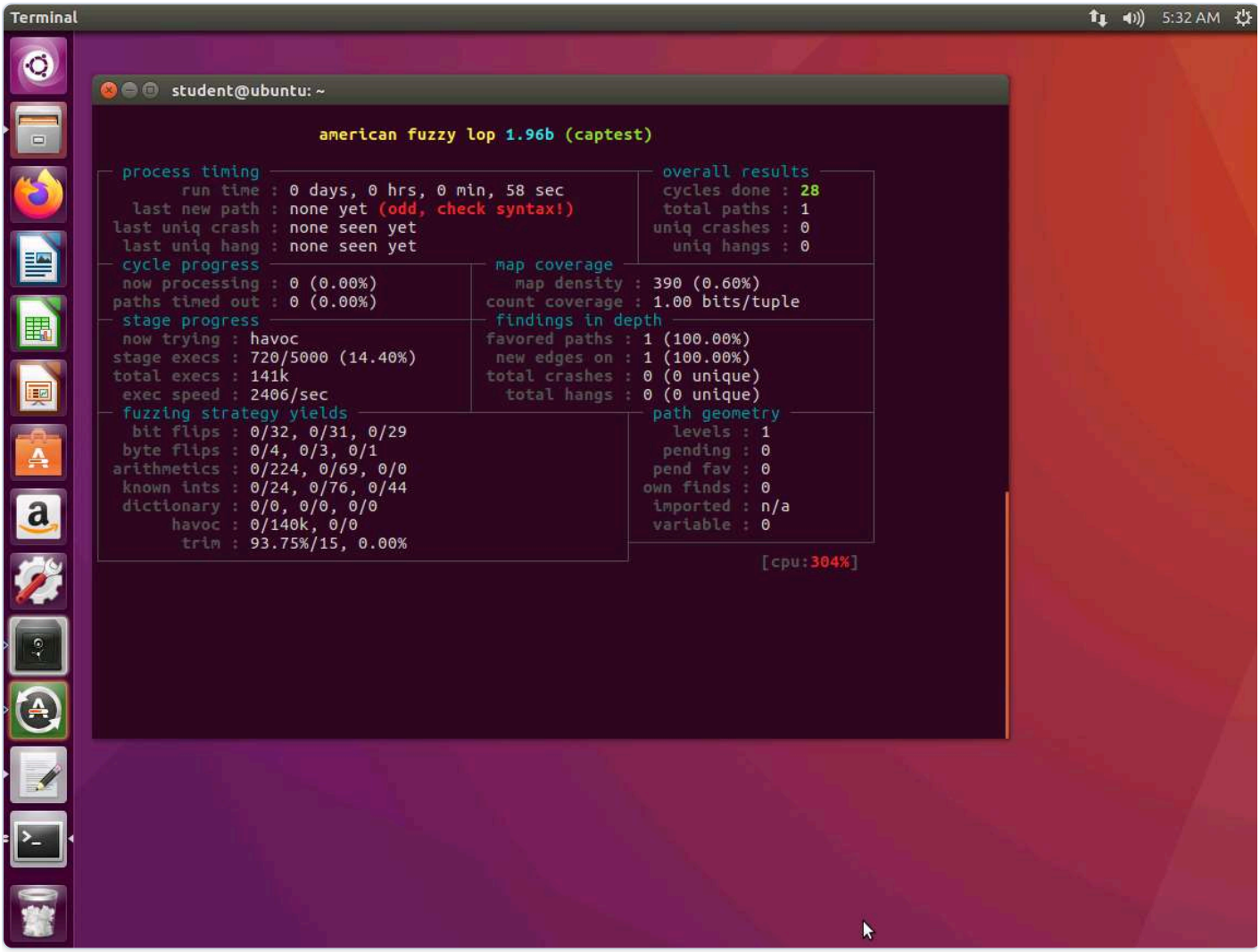
   echo > /proc/sys/kernel/core_pattern



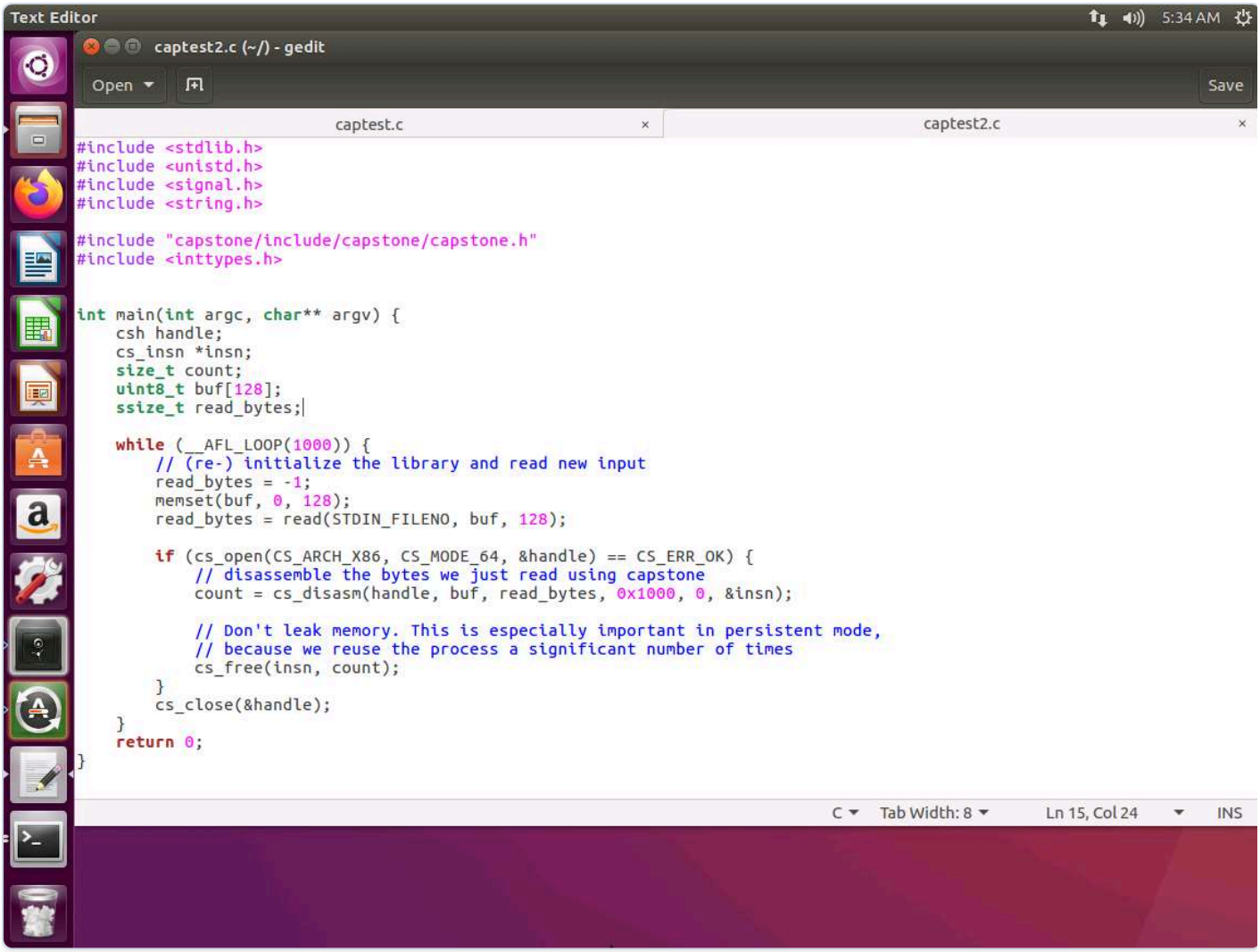
1. Execute baseline AFL fuzzing session:

   afl-fuzz -i inputs -o findings ./captest

   Observe execution speed: ~2k execs/sec.

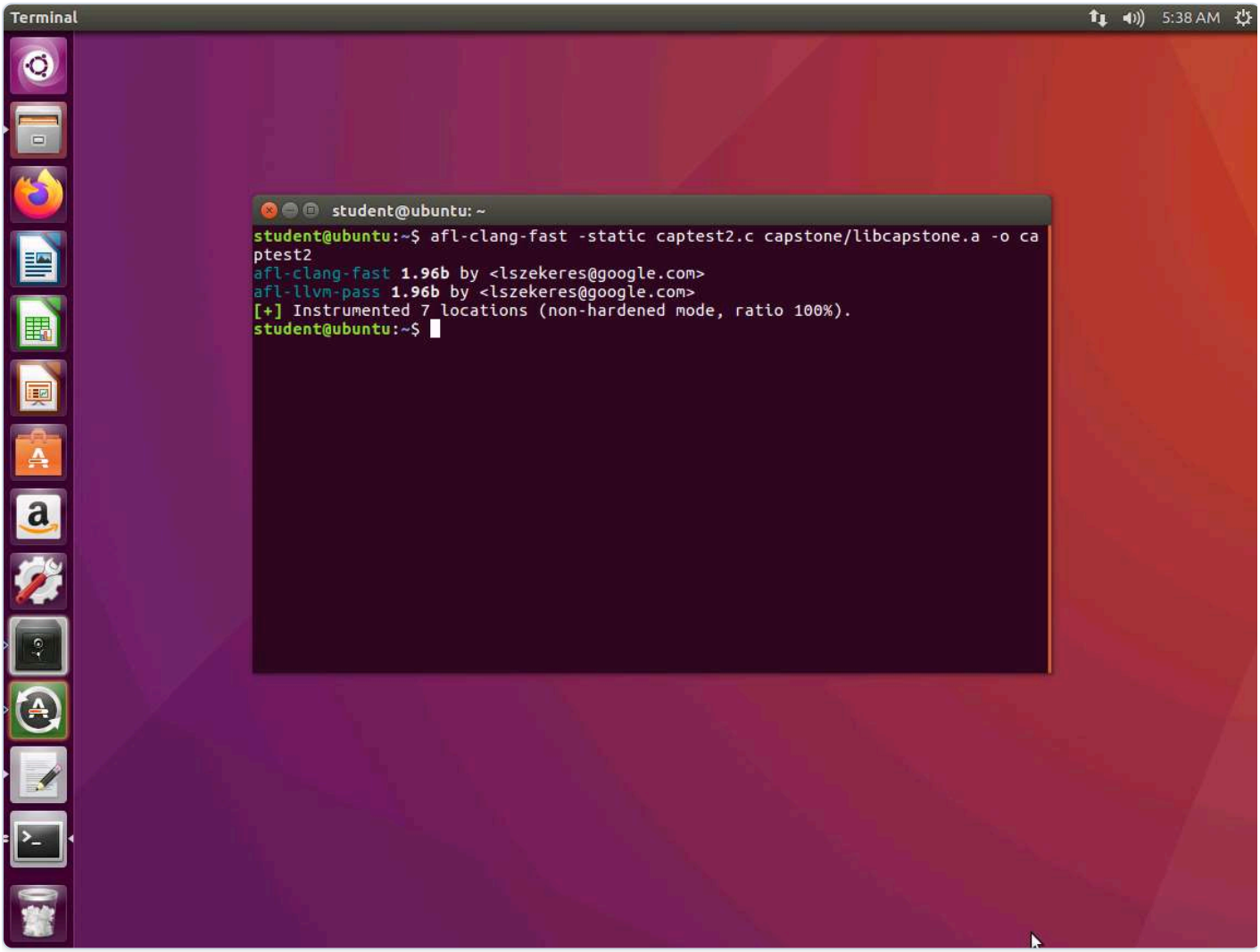


1. Inspect persistent in-process looping source code **captest2.c** containing `__AFL_LOOP(1000)`.



1. Compile persistent loop target binary:

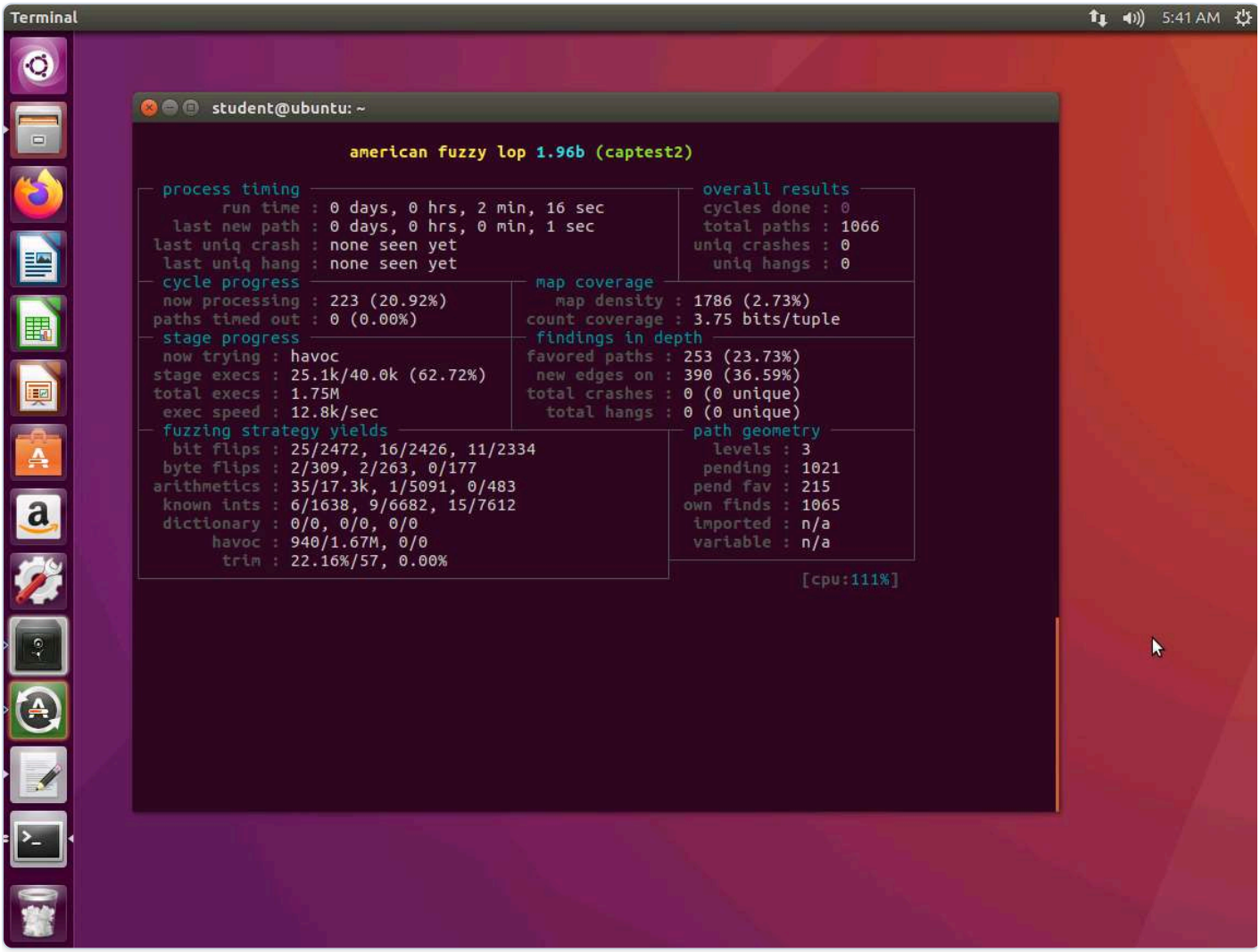
   afl-clang-fast -static captest2.c capstone/libcapstone.a -o captest2



1. Run optimized AFL fuzzing session:

   afl-fuzz -i inputs -o findings ./captest2

   Observe execution speed: **12.8k/sec** (12,800 execs/sec across 1.75M total execs).



This concludes the lab exercise.

IMPORTANT: ❓ **Question 11.15.1.1** Perform fuzzing using American Fuzzy Lop (AFL) and the capstone tool in the Ub20 Fuzzing-CAP machine. The sample code (captest.c) is available in the home directory. Enter the exec speed identified after performing fuzzing without making changes to the sample code. [!SUCCESS] ✅ **Answer:** **12.8k/sec**